



**UNIVERSITÀ
DEGLI STUDI
DI TRIESTE**

Solving Chess Endgames

A Reinforcement Learning Approach

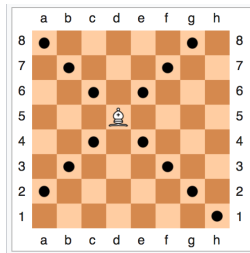
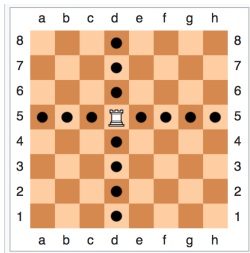
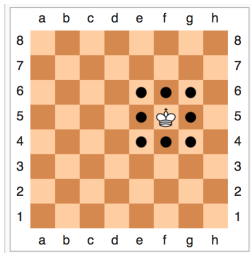
Valeria De Stasio, Christian Faccio, Giovanni Lucarelli

September 5, 2025

Chess Programming Background

- 1890 El Ajedrecista: electro-mechanical KRK solver
- 1950 Claude Shannon: first article about chess programming
- 1997 *Deep Blue*: first computer program to beat the current world champion
- 2008 *Stockfish*: strong heuristic-based chess engine
- 2017 *Alphazero*: neural network-based approach that defeated Stockfish
- from 2018: more models developed based on *Alphazero*

Chess Rules



- **Win:** checkmate the opponent's king
- **Draw:** stalemate or insufficient material¹

¹threefold repetition and the fifty-move rule are not considered in our implementation

Elementary Endgames

- subset of endgames with few pieces left
- clear theoretical results: forced checkmate or draw
- foundation for learning more complex endgames
- examples: KRK, KQK, KBBK
- solved with tablebases up to 7 pieces

MDP Formulation

Markov Game vs MDP

- Chess is a 2-player, turn-based, zero-sum game, formally a Markov Game
- Assume fixed black defender policy: **oracle** or **random**
- Black becomes part of the environment:
Markov Game $\rightarrow$ MDP

Remark: white is always winning in elementary endgames!

MDP: States

- State space $\mathcal{S}$: legal pieces positions and side-to-move

```
8/8/K7/8/8/5k2/7R/8 w - - 0 1
```

```
std::array<std::array<U64, 6>, 2> pieces;  
Color side_to_move;
```

- Terminal state space $\bar{\mathcal{S}} \subset \mathcal{S}$: checkmate or draw positions (and stm). Defined procedurally

```
state.is_game_over() -> bool
```


- Action space $\mathcal{A}(s)$: set of legal moves for the current player.
- Defined procedurally:

```
state.legal_moves() -> std::vector<Move>
```

Transition probability $\Pr(S_{t+1} = s' | S_t = s, A_t = a)$

- **Oracle Defender:** deterministic evolution

```
state.do_move(Move)           // white deterministic
state.do_move(oracle.reply()) // black deterministic
```

- **Random Defender:** stochastic evolution

```
state.do_move(Move)           // white deterministic
state.do_move(random_reply()) // black random
```

Goal: find the fastest mate

Reward:

- checkmate: +1
- draw: -1 (always winning for white)
- encourage faster mates
 - step penalty: -2
 - discount factor $\gamma = 0.99$

RL Algorithms

Monte Carlo Tree Search

- One of the oldest methods to solve chess
- Explore the game tree by simulating moves

Dynamic Programming

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop:

```
|  $\Delta \leftarrow 0$   
| Loop for each  $s \in \mathcal{S}$ :  
|    $v \leftarrow V(s)$   
|    $V(s) \leftarrow \max_a \sum_{s',r} p(s', r | s, a) [r + \gamma V(s')]$   
|    $\Delta \leftarrow \max(\Delta, |v - V(s)|)$   
until  $\Delta < \theta$ 
```

Output a deterministic policy, $\pi \approx \pi_*$, such that
$$\pi(s) = \operatorname{argmax}_a \sum_{s',r} p(s', r | s, a) [r + \gamma V(s')]$$

- KRk: 182.676 states
- KQk: 152.968 states
- KBBk: ? states

Choice of parameters

- Step penalty: $-2 \rightarrow$ values = DTZ
- Draw penalty: $-100 \rightarrow$ slow mates > quick draws
- $\gamma = 1$
- $\theta = 0.0001$

Strong assumption: opponent plays optimally! Idea: train against an opponent that plays at random and see how well it can play against an optimal one.

Q-Learning: model-free approach

Assume no information on transitions!

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

 Initialize S

 Loop for each step of episode:

 Choose A from S using policy derived from Q (e.g., ε -greedy)

 Take action A , observe R, S'

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

$S \leftarrow S'$

 until S is terminal

Off-policy: we don't care about the behavior policy!

Choice of parameters

- Max steps per episode: 50 $\rightarrow$ if it's too far from mate, agent may never reach terminal state
- Step penalty: 0 $\rightarrow$ otherwise would learn wrong values from early termination
- Draw penalty: -1
- $\gamma = 0.99$
- Learning rate $\alpha = 0.05$
- ϵ -greedy policy with $\epsilon = 0.3$ with decay

Full Reinforcement Learning

- Problem: too many states, **curse of dimensionality!**
- Idea: use neural networks for policy and linear approximation for value function

One-step Actor–Critic (episodic), for estimating $\pi_{\theta} \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s, \theta)$

Input: a differentiable state-value function parameterization $\hat{v}(s, \mathbf{w})$

Parameters: step sizes $\alpha^{\theta} > 0$, $\alpha^{\mathbf{w}} > 0$

Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^d$ (e.g., to $\mathbf{0}$)

Loop (for each episode):

 Initialize S (first state of episode)

 Loop while S is not terminal (for each time step):

$A \sim \pi(\cdot|S, \theta)$

 Take action A , observe S', R

$\delta \leftarrow R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})$ (if S' is terminal, then $\hat{v}(S', \mathbf{w}) \doteq 0$)

$\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla \hat{v}(S, \mathbf{w})$

$\theta \leftarrow \theta + \alpha^{\theta} \delta \nabla \ln \pi(A|S, \theta)$

$S \leftarrow S'$

Approximations

- Policy function approximation: neural network
- State space approximation: linear combination of features
- Feature engineering:
 - Manhattan distance between kings
 - Manhattan distance between black king and white rook
 - File and rank of the kings
 - Distance of the Black King from the side of the board
 - Distance of the black king from the corner of the board
 - Binary indicator of opposition

Results

Assessment Metrics Overview

* DTM, DTZ etc * success, top1 etc

Why no convergence?

- Difficulties:
 - too many possible actions at each step
 - ideal opponent that never makes mistakes
 - very sparse rewards in an episode
- Possible solutions:
 - more training time
 - implement draw by repetition and 50-move rule → more frequent terminal states

Policy interpretability through human chess principles

Video Animation of optimal ply vs our policy

maybe for a mate in 5 or more

Future Works

- * zobrist hashing and invariance properties in chess endgames to reduce state space cardinality

Why is it so difficult to solve chess endgames?

A player can sometimes afford the luxury of an inaccurate move, or even a definite error, in the opening or middlegame without necessarily obtaining a lost position. In the endgame ... an error can be decisive, and we are rarely presented with a second chance.

- Paul Keres

Thank You!

Why $\gamma = 0.99$?

The longest rkr endgame requires 16 white moves, hence $\gamma^{16} \approx 0.85$ is a good reward for reaching the goal and producing fast mates.